CompSci 471

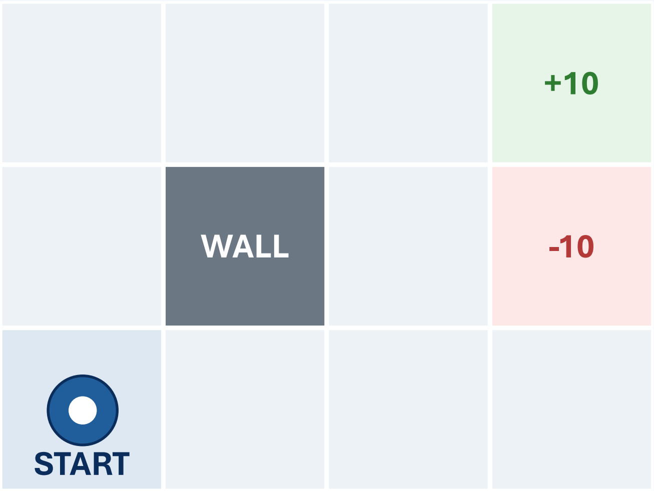
Artificial Intelligence

Lesson 11: MDP Foundations

Modeling decisions under uncertainty

State → Action → Transition → Reward → New state

Shared Gridworld Lab



Build the world before solving it.

Fall 2026

Module 2 map

One decision model. Three ways to reason with it.

L11	L12	L13	L14	L15	L16
Build the MDP world	Behavior, return, value	Value Iteration	Policy Iteration	Q-Learning	Workshop + Quiz 4

Today: describe the environment before asking an algorithm to solve it.

Today’s mission

A strong MDP formulation separates what the agent knows, chooses, experiences, and values.

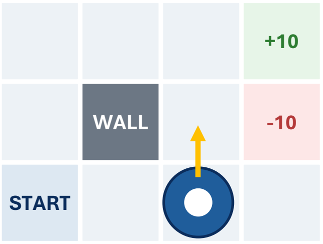
1. **Define states, actions, transitions, and rewards.**
2. **Explain the Markov property.**
3. **Formulate a real-world problem as an MDP.**

Performance target: given a new scenario, build S, A, T, R and explain whether the state contains enough information to predict what happens next.

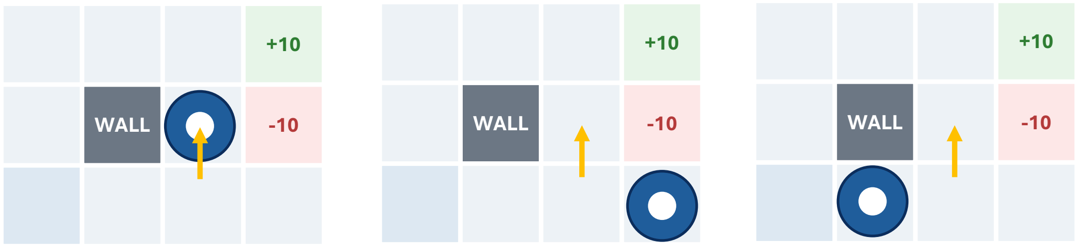
From search to MDPs

We are extending problem formulation, not replacing it.

Search	Expectimax	MDP
Action usually has one modeled result Solution: path	Action is chance with several possible outcomes Solution: policy	Action may have several possible resulting states Solution: policy



one planned path



one action, multiple outcomes

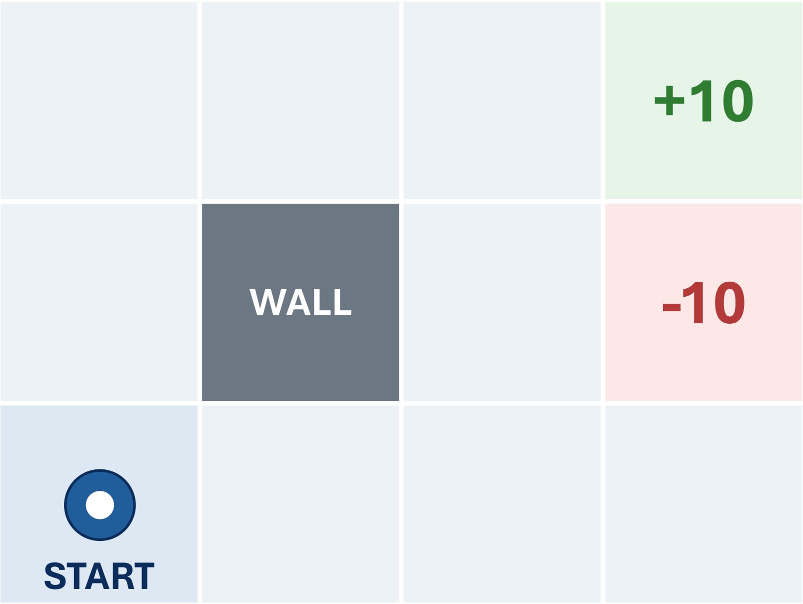
outcome distribution

s'	P(s'|s,a)
(1,2)	0.15
(0,3)	0.6
(0,1)	0.25

Key question: What if taking the same action does not always produce the same next state?

Shared Gridworld Lab

We will reuse one small world to make each algorithm visible.



Environment

- Agent starts in the lower-left cell
- Gray cell is a wall
- Green terminal gives +10
- Red terminal gives -10

Actions

↑ ↓ ← →

Today: identify the model. Later: compute or learn good behavior.

What information does an MDP need?

Every later algorithm assumes these pieces are clearly defined.

State

current situation

Action

agent choice

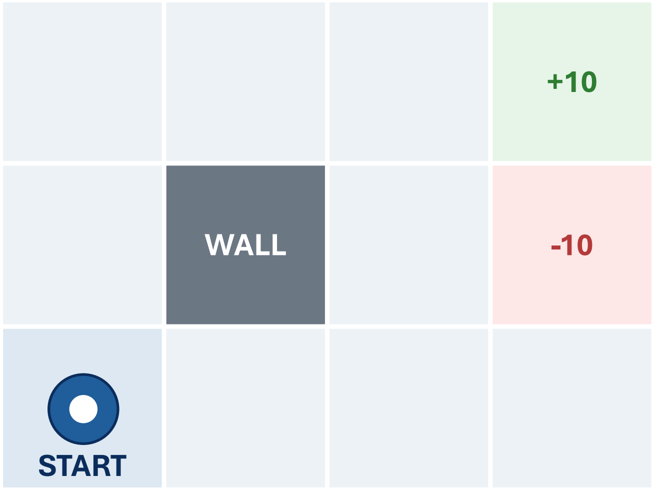
S, A, T, R

Transition

possible next states
+ probabilities

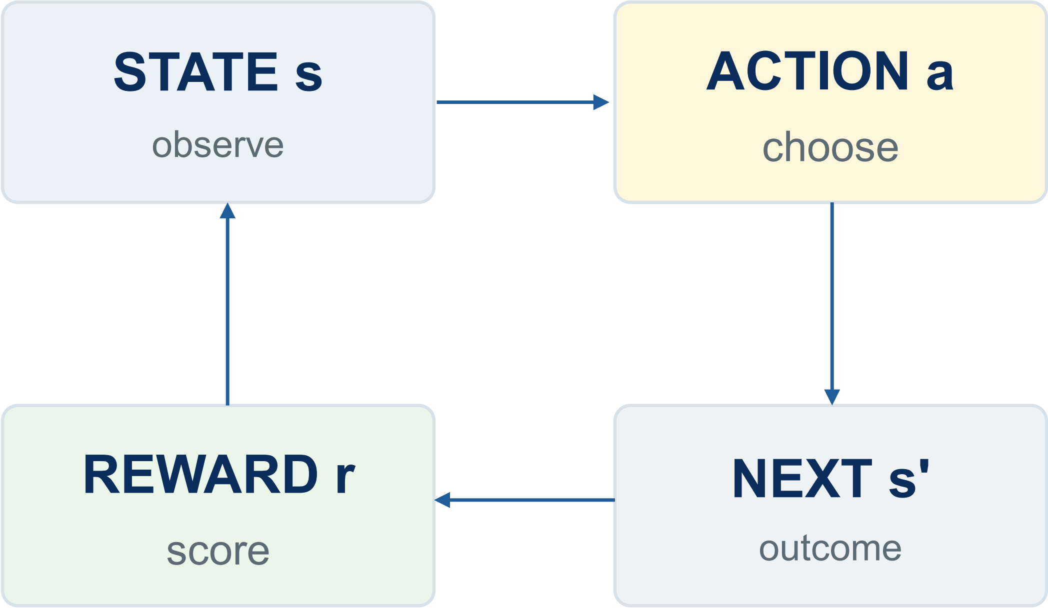
Reward

immediate score



The MDP interaction loop

The agent chooses the action; the environment determines the outcome.



Agent controls

which action is selected from the current state

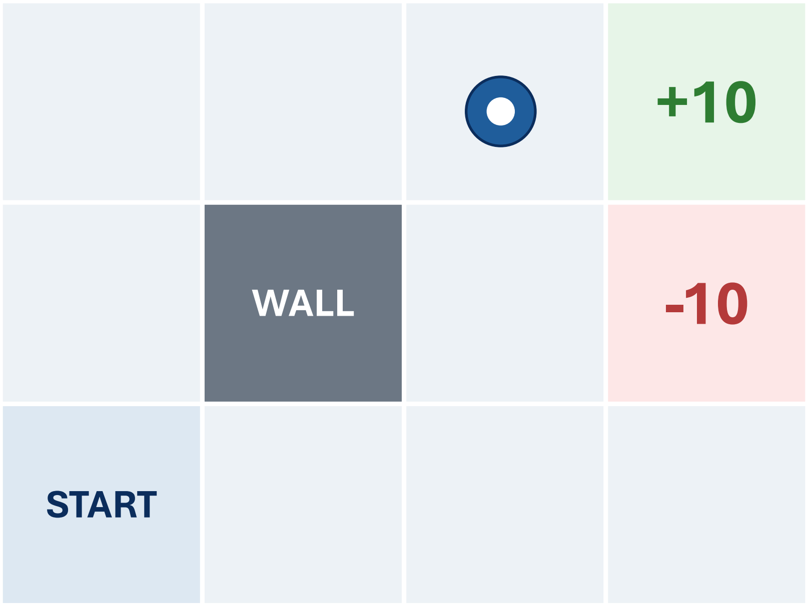
Environment responds

next state and reward according to the **transition model**

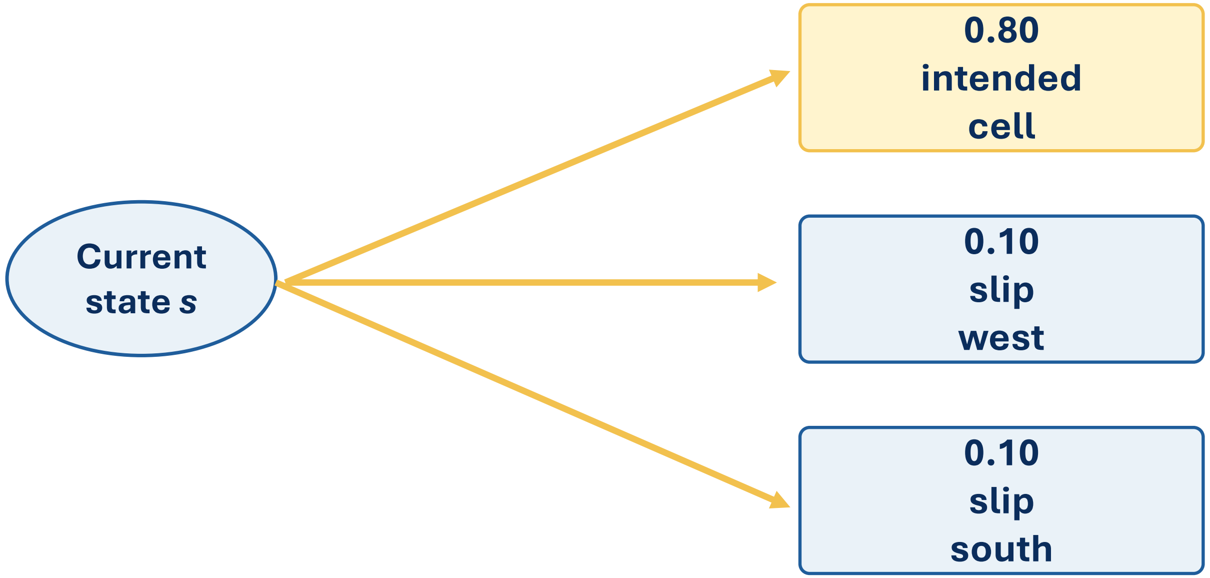
A policy will eventually tell the agent what action to choose in each state.

Probability checkpoint: one action, several outcomes

We only need enough probability to interpret uncertain transitions.



Action chosen: → East



Probabilities for a complete set of outcomes must add to 1.

Reading transition notation

The notation is just a compact way to name a conditional probability.

$$P(s' \mid s, a)$$

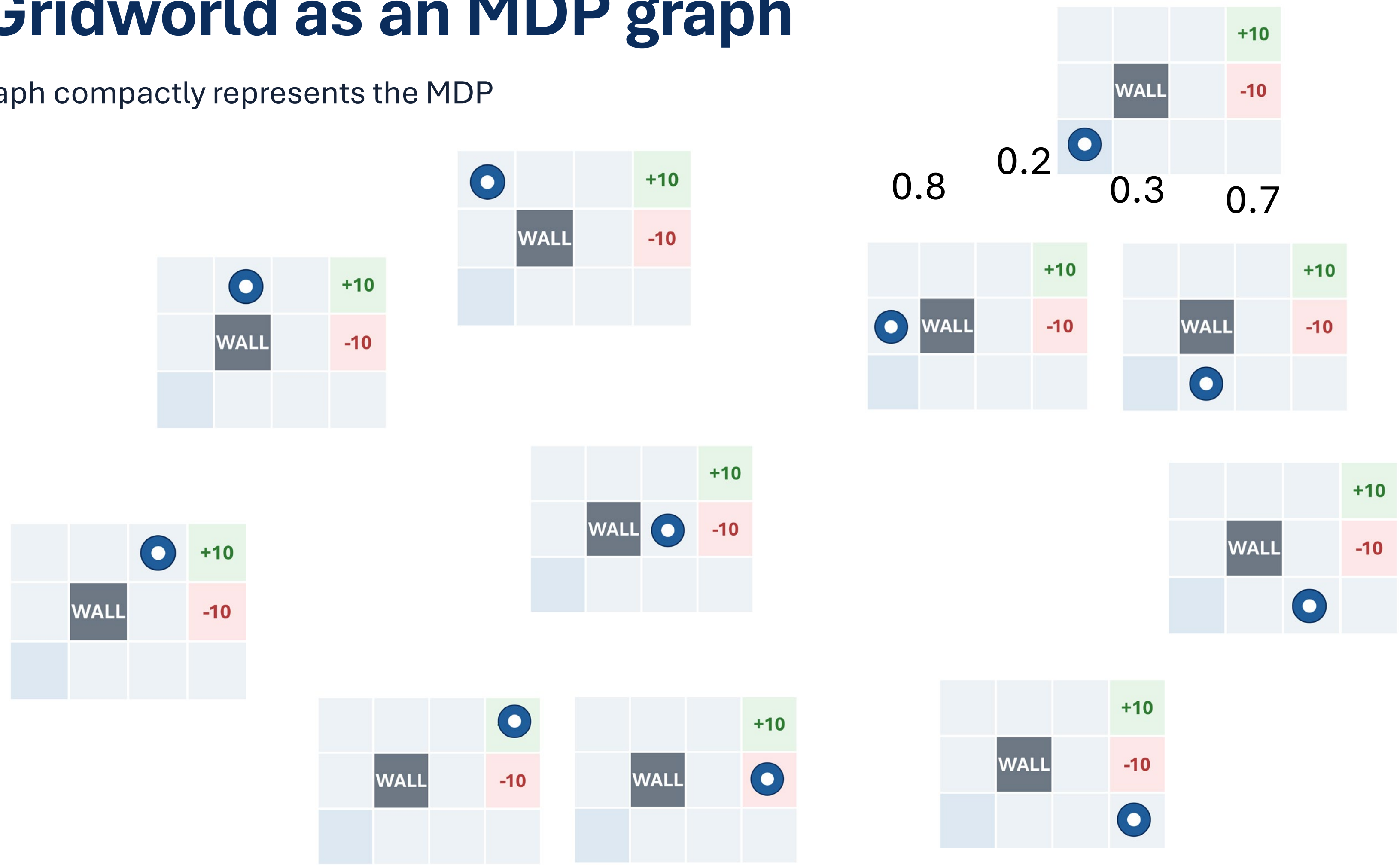
s	a	s'	$\mid$
current state	chosen action	possible next state	given

Probability of reaching next state s', given current state s and action a.”

Formal conditional probability returns in Lesson 18.

The Gridworld as an MDP graph

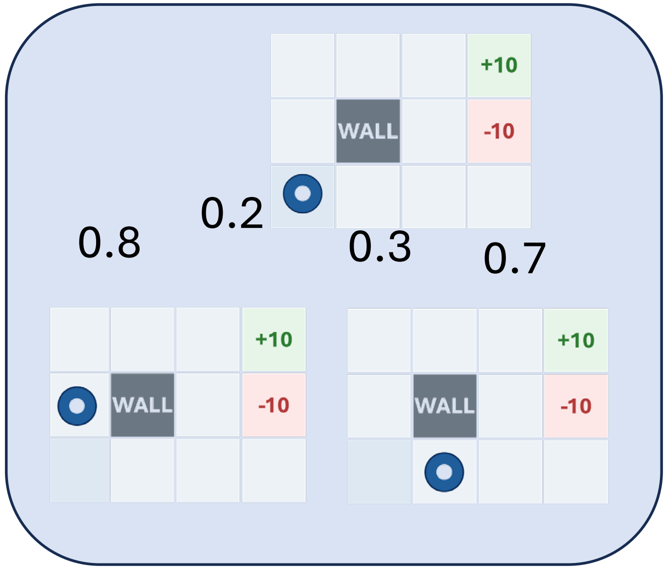
A state graph compactly represents the MDP

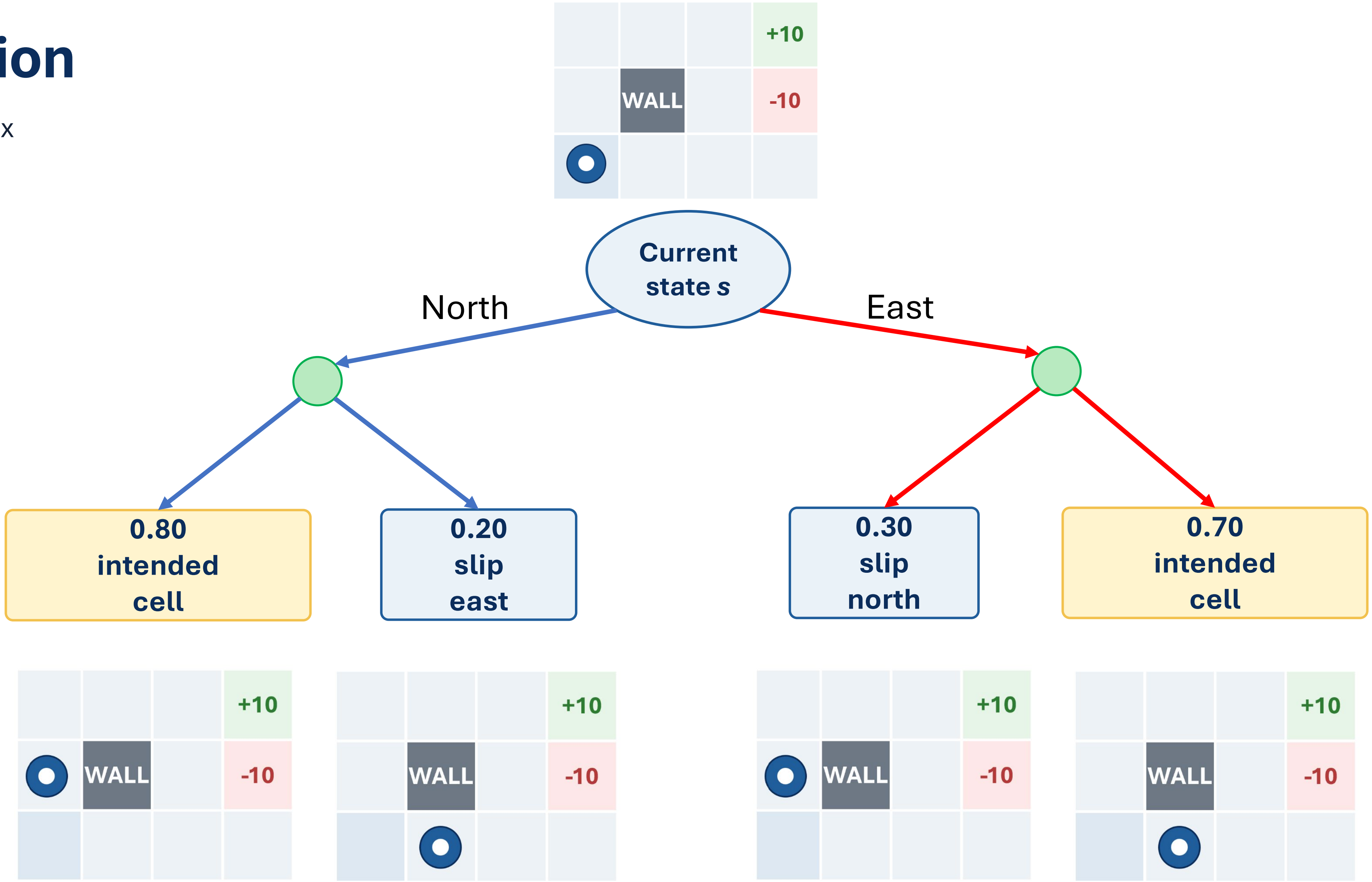


This is the world model—not a record of one particular run.

Unrolling a decision

This tree looks similar to expectimax

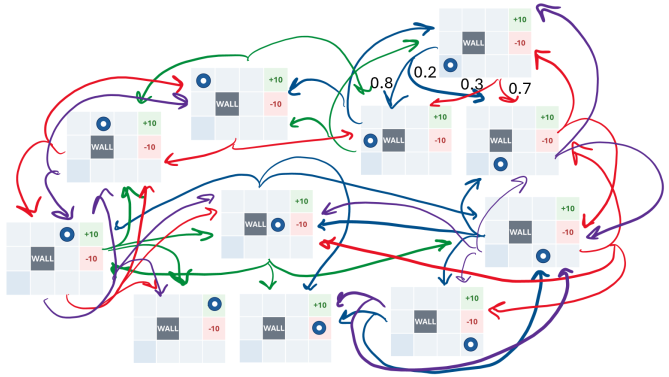


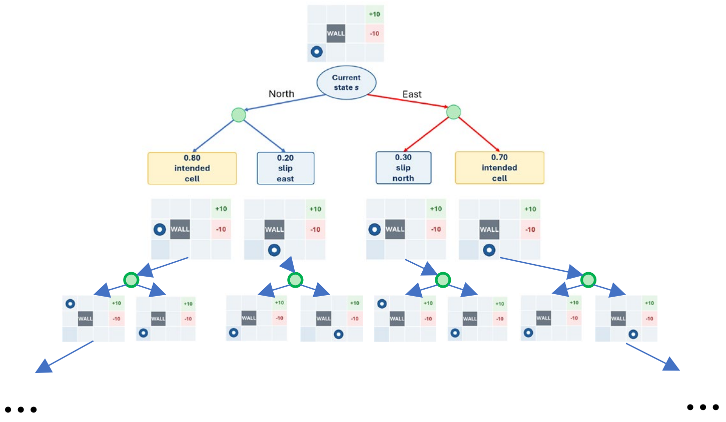


The local decision structure is familiar. What changes is that an MDP keeps going.

Graph vs. tree

The graph stores the problem; the tree unfolds possibilities through time





Compact model

- One node per state
- States can be revisited
- Cycles are natural
- Reusable across decisions

Unrolled futures

- Same state may appear many times
- Depth represents future decisions
- Tree grows rapidly
- Finite lookahead requires a horizon

MDP methods reuse what we learn about a state instead of solving an enormous new tree every time we encounter it.

The four MDP components

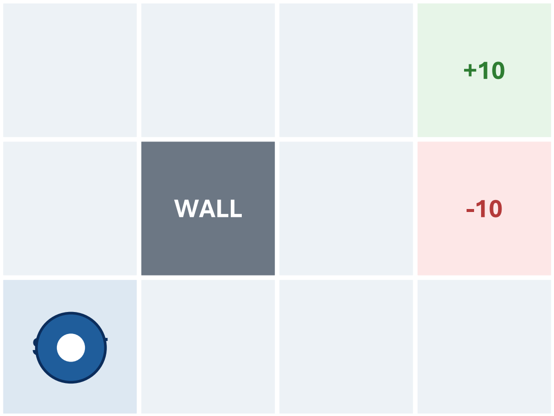
Use these questions to formulate any sequential decision problem.

State S	Transition T
What situation are we in now?	What might happen next?

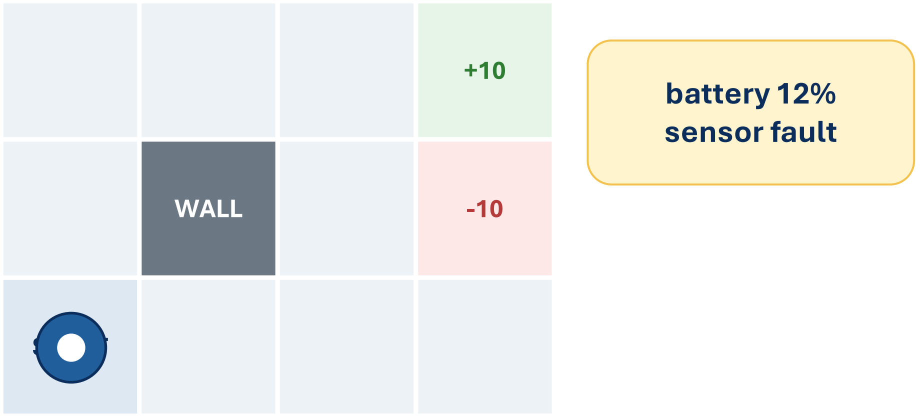
Action A	Reward R
What can the agent choose?	How desirable was the immediate outcome?

State = decision-relevant snapshot

A state should contain information needed for the next decision.



s = (row, column)



s = (row, column, battery, sensor)

In a simple Gridworld,
location may be enough.

In real systems,
hidden variables may change transitions or rewards.

State representation still matters

Too little information breaks prediction; too much information adds needless complexity.

Too little	Useful abstraction	Too much
“operating normally” Cannot predict fuel, demand, or risk.	battery, generator, demand, solar condition Supports decisions and transition prediction.	every sensor reading, serial number, operator name May add complexity without changing decisions.

MDP state must support prediction, not just description.

Actions are choices, not conditions

Separate what the agent chooses from what the environment returns.

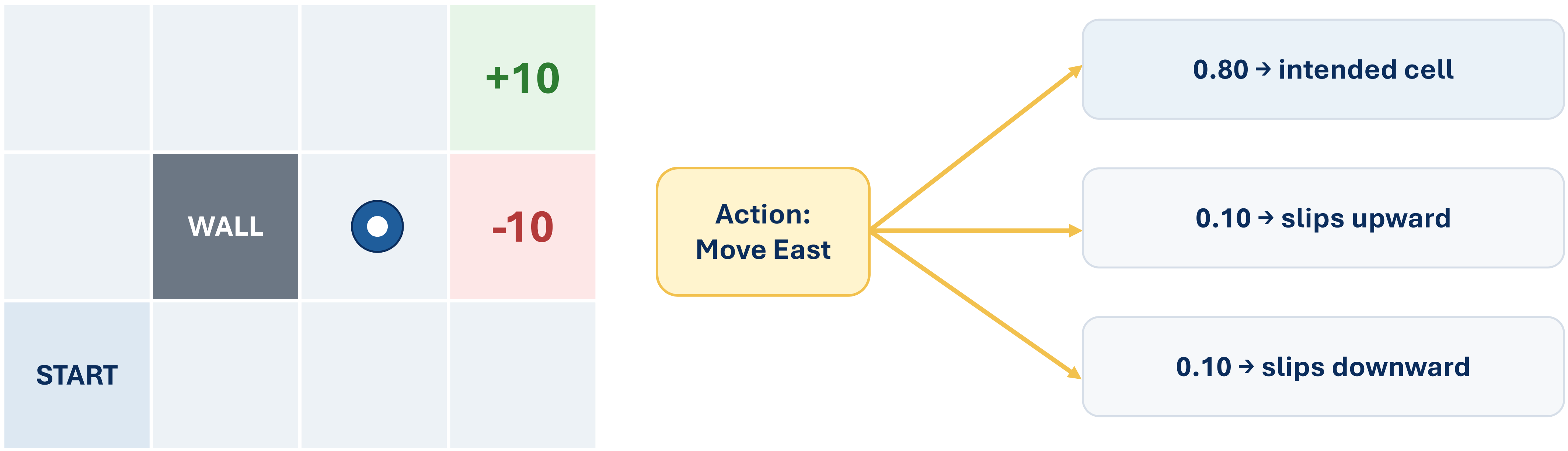
Action	Not an action
Move East	Battery at 20%
Start generator	0.30 probability of failure
Shed noncritical load	+10 reward
Transmit data	Generator starts successfully

Action = what the agent can intentionally select.

Transition model = possible outcomes + probabilities

The action is one choice; the transition is a distribution over next states.

$$P(s' \mid s, a)$$

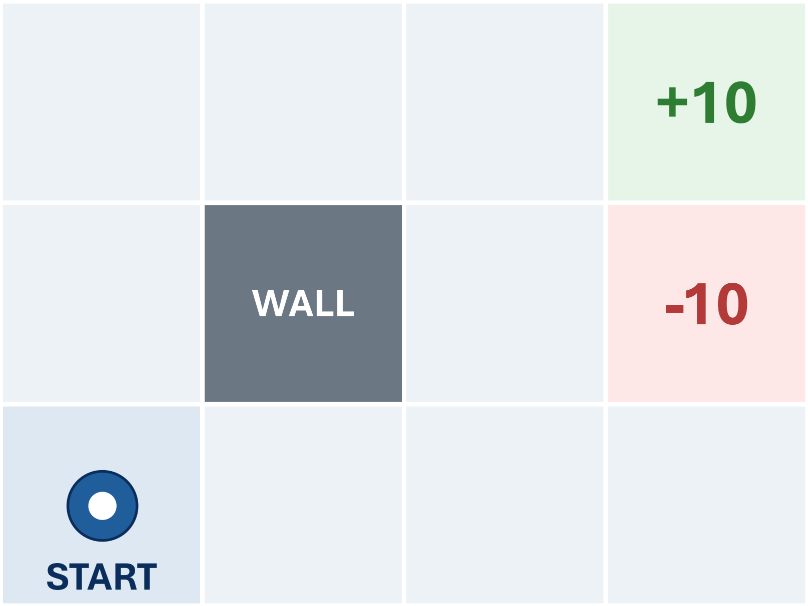


Action chosen: → East

Transition probabilities describe uncertainty in what happens next.

Rewards score immediate outcomes

Reward is the numerical consequence attached to what just happened.



Positive

+10 for reaching the mission goal

Negative

-10 for entering the hazard

Living reward

-1 each time step to encourage efficiency

Immediate reward is not the same thing as total long-term value.

Transition tells what happened; reward tells how we score it

Keep the event and the evaluation separate.

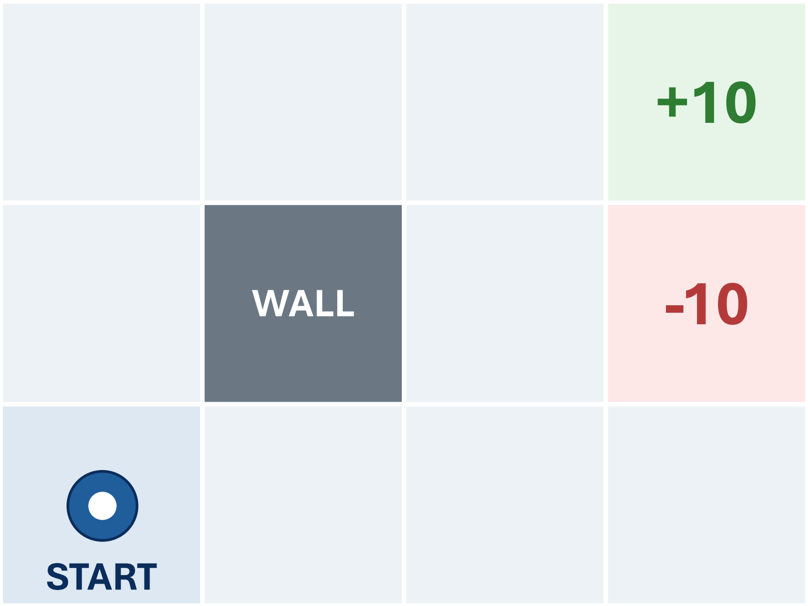
Action: Move East

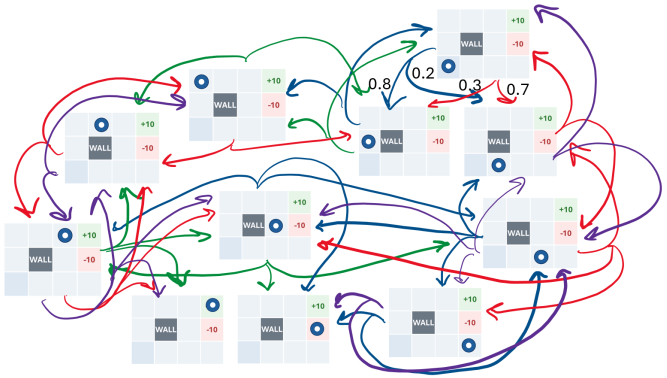
Probability	Transition outcome	Reward
80%	arrive in intended cell	**-1**
10%	slip near hazard	**-1**
10%	slip into hazard	**-10**

Later algorithms combine these pieces into long-term value.

Assemble the Gridworld MDP

A complete formulation lets later algorithms reason consistently.





S grid cells plus terminal states

A ↑, ↓, ←, → when legal

T 1 – (0.1 * n) intended, 0.1 * n slip one of n directions

R +10 goal, -10 hazard, -1 living reward

Formulation is an engineering design choice.

Transfer: expeditionary microgrid controller

The same MDP vocabulary applies outside Gridworld.

State	Actions	Transitions	Rewards
battery level generator status demand solar condition	use battery start generator charge shed load	clouds arrive demand spikes generator fails battery drains	mission power fuel cost safety outage penalty

Gridworld teaches the structure;
mission systems show why the structure matters.

The Markov question

Does the current state tell us enough to predict what happens next?

Recorded state: “generator is OFF”

Situation A	Situation B
fuel high generator cool battery healthy	fuel nearly empty generator overheated battery low

Same recorded state + same action can produce different transition probabilities.

The Markov property

Relevant history should be summarized into the present state.

Once the current state and action are known,
additional history should not be needed to predict the next-state distribution and immediate reward.

$$P(s_{t+1}, r_{t+1} \mid s_t, a_t, \text{history}) = P(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

Markov does not mean the past never matters.
It means the relevant effects of the past are encoded now.

Repair the state, not the definition

If hidden history changes outcomes, add the variables that carry its effects.

Incomplete state

s = (location)

→

Improved state

s = (location, battery, sensor status, mission deadline)

Reason

Battery and sensor status change which actions are available, which outcomes are likely, and how rewards should be scored.

Add information because it changes prediction—not because it is merely available.

The Markov test

Use this checklist when a proposed state representation seems too simple.

1. What actions are available?
2. After an action, what next states are possible?
3. Can we assign appropriate transition probabilities?
4. Can we determine the immediate reward?
5. **Would additional history change any answer above?**

If #5 is yes, the state may be incomplete.

Practice: debug the MDP

Find the category errors before trying to solve the problem.

Pairs: repair the formulation and decide whether “current field” is Markov.

Autonomous farming robot

State	current field
Action	current weather
Transition	reward +5
Reward	crop becomes healthier

- Weather is not an action.
- Reward should be numerical.
- Transition describes resulting states.
- State may need soil moisture and forecast.

Practice: MDP formulation sprint

Build a compact model from a new scenario.

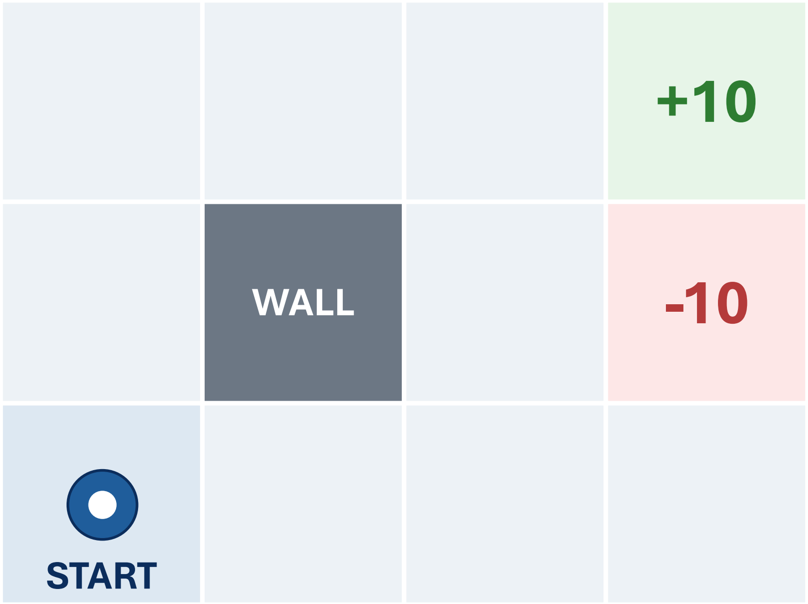
A coastal monitoring buoy decides whether to transmit data, conserve power, or run diagnostics. Battery charge, weather, and sensor status affect the result.

S	reasonable state
A	two actions
T	one action with two outcomes + probabilities
R	one positive and one negative reward
M	one Markov-property sentence

Goal: a model clear enough that another team could evaluate it.

Synthesis: what did we build?

An MDP is the decision environment that later algorithms solve or learn from.



S state = ____________________

A action = ____________________

T transition = ____________________

R reward = ____________________

M Markov state = ____________________

Next: once the environment is modeled, how do we describe good behavior over time?

Exit ticket

Complete before you leave.

Scenario

A coastal monitoring buoy controller decides whether to transmit data, conserve power, or perform diagnostics.

Battery charge, weather, and sensor status affect the result.

One reasonable state representation is... ________________

One action available to the agent is... ________________

One possible outcome of that action is... ________________

One immediate reward could be... ________________

This state is / is not Markov because... ________________

Preview L12: policies, return, and value.